Presentazione

Natile Martino Andrea
Romani Pierluigi

Project goals

Project goals

- Theoretical study of the **kNN** baseline
- Analysis and implementation of **ANN** models
- Evaluation of **Approximation Severity**
- Multi-Objective Optimization Analysis
- Development of an extensive visualization framework

Neighbor-Based Approach

kNN and Collaborative Filtering

If two users agreed on past items, they will likely agree on future items recommendations.

User-based

People **like you** also liked this item

Users with similar item rating history are **neighbors**

Item-based

Since you liked this item, you will like this **similar one**

Items bought by similar sets of users are **neighbors**

kNN and Collaborative Filtering

Find ***k*** similar items based on their similarity (jaccard, cosine)

Highly accurate – gold standard

Computationally expensive – $O(N^2)$

- must compute similarity of an item with every other one

ANN models

From mathematically closest to close enough

Trade a drop in accuracy for a significant improvement in efficiency

Compute similarity of a subset of items

ANN models

LSH	Group similar items into "buckets" using hash functions MinHashing (jaccard) or Random Projection (cosine)
Faiss	Developed by Meta Clusters space into *Voronoi Cells* Select closest centroid to the query and search in its cell

ANN models

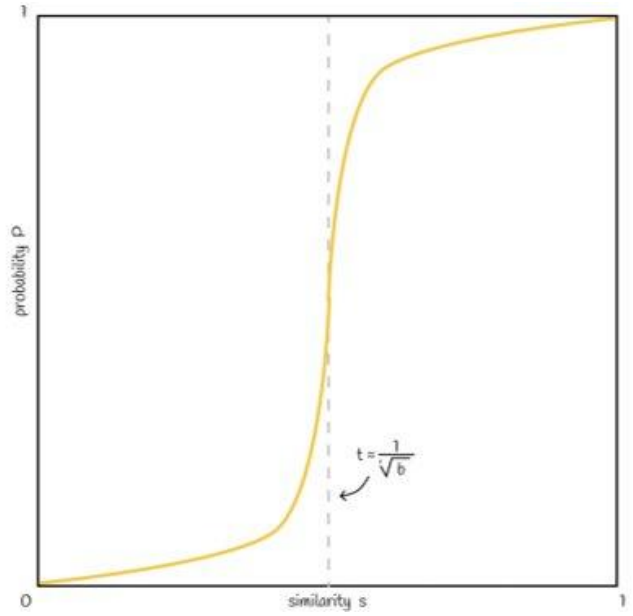
Annoy	Developed by Spotify Slice space with random hyperplanes and builds forests of binary trees
FairANN	Stems from standard LSH Picks candidates and applies **sampling** strategy to achieve **fairness**

Approximation Severity

Threshold and S-Curve

When the similarity of two items is above a **target threshold**, they are considered candidate pairs.

The **s-curve** models the probability of collision – how **similar** must two item be to end up in the **same bucket**



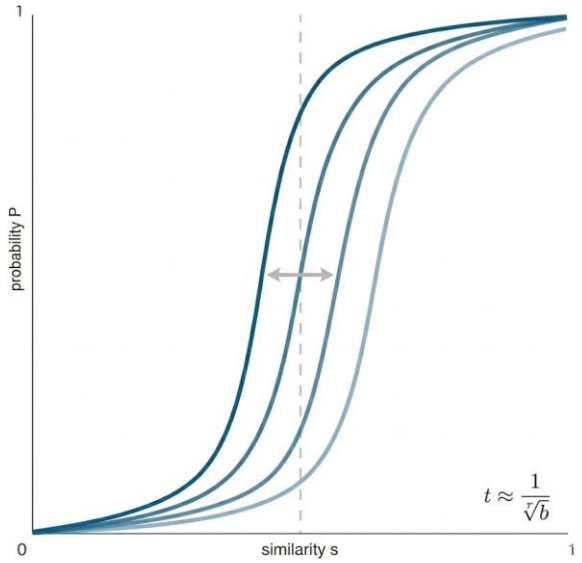
We can control the perfomances by moving the **inflection point** – i.e. the **similarity threshold** – by tuning the model parameters

- k: **AND** matching
- L: **OR** matching

Approximation Severity

Each ANN model has its own parameters

The similarity thresholds acts as a baseline to compare these different models



Higher threshold → **strict** approximation

Lower threshold → **loose** approximation

Hyperparameters Tuning

The Probability of two items having a similarity of s of being in the same bucket

$$P = 1 - (1 - s^k)^L$$

$$t \sim \frac{1}{L}^{\frac{1}{k}}$$

Jaccard Similarity

$$P = Similarity_{jaccard}$$

$$L = \frac{1}{t}^{k}$$

Cosine Similarity

$$P = 1 - \frac{arccos(t)}{\pi}$$

$$L = \frac{1}{P}^{k}$$

Research Questions

Research Questions

RQ1	**The Accuracy-Efficiency Trade-off**
RQ2	**Impact on Beyond-Accuracy (Diversity and Novelty) and (Provider) Fairness**
RQ3	**Pareto Dominance in Multi-Objective Scenarios**

Experimental Setup

- All the experiments were done using the Movielens100k dataset
- All the models have the same experimental setup:
- **user_k_core**: 20
- **Top k**: 50
- **Cutoff** at 20
- Three different **number of neighbors** [50, 100, 250]
- **Jaccard** and **Cosine** similarity (**Angular** for Annoy)
- 5 different levels of **severity approximation**
  [0.05, 0,25, 0.50, 0.75, 0.95]

Results

What we were expecting

With the introduction of **approximation**

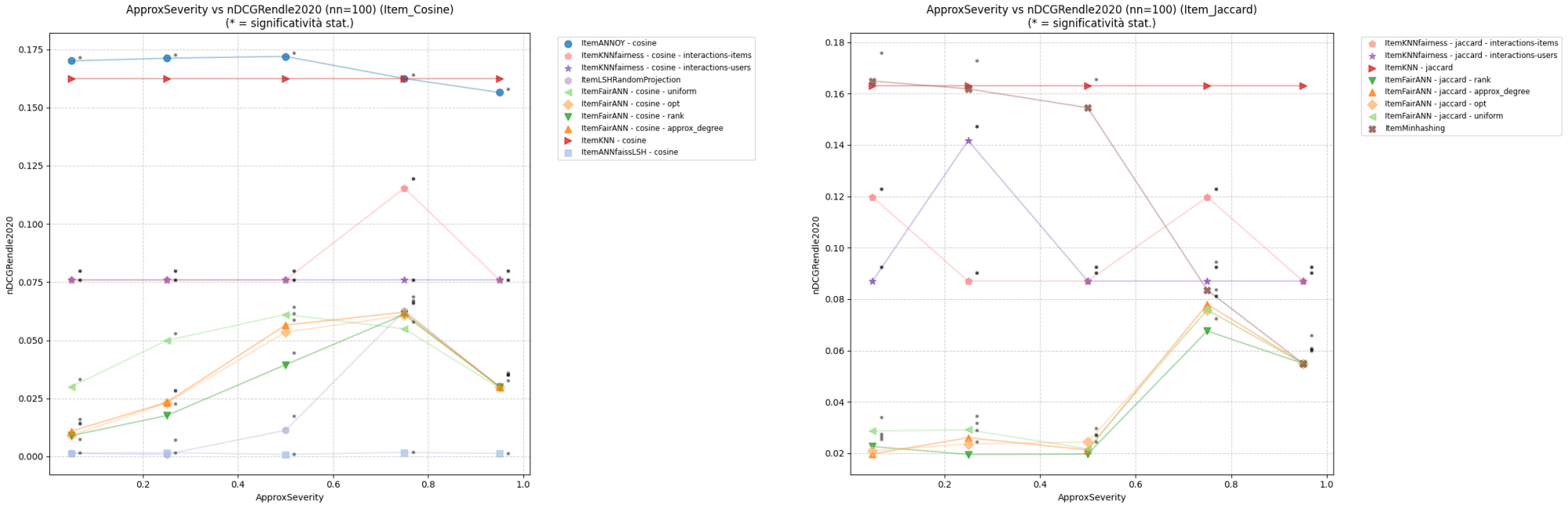
Drop in accuracy

Improved efficiency

Better beyond-accuracy

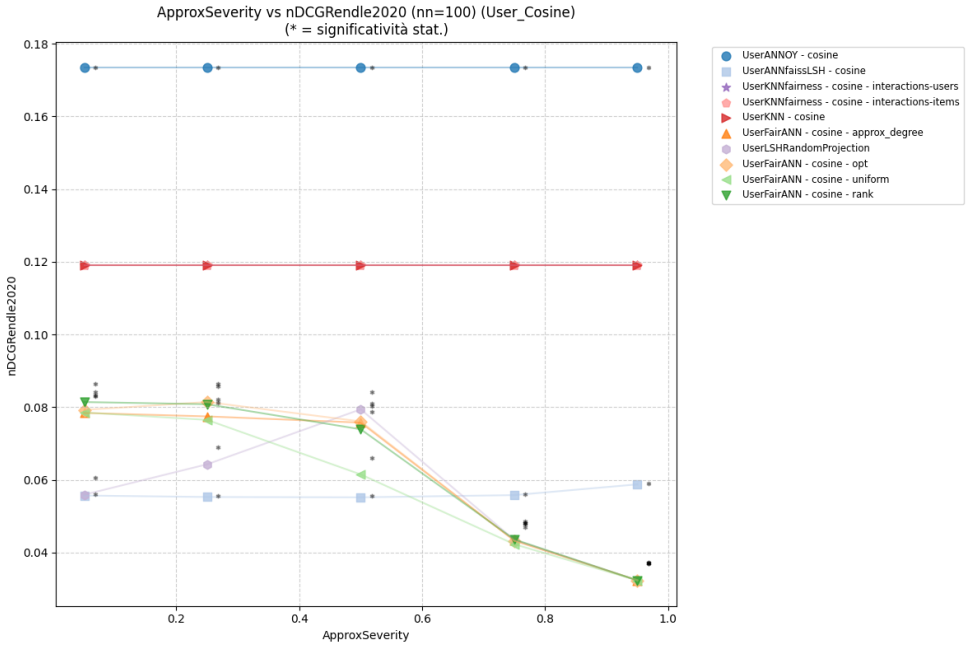
RQ1

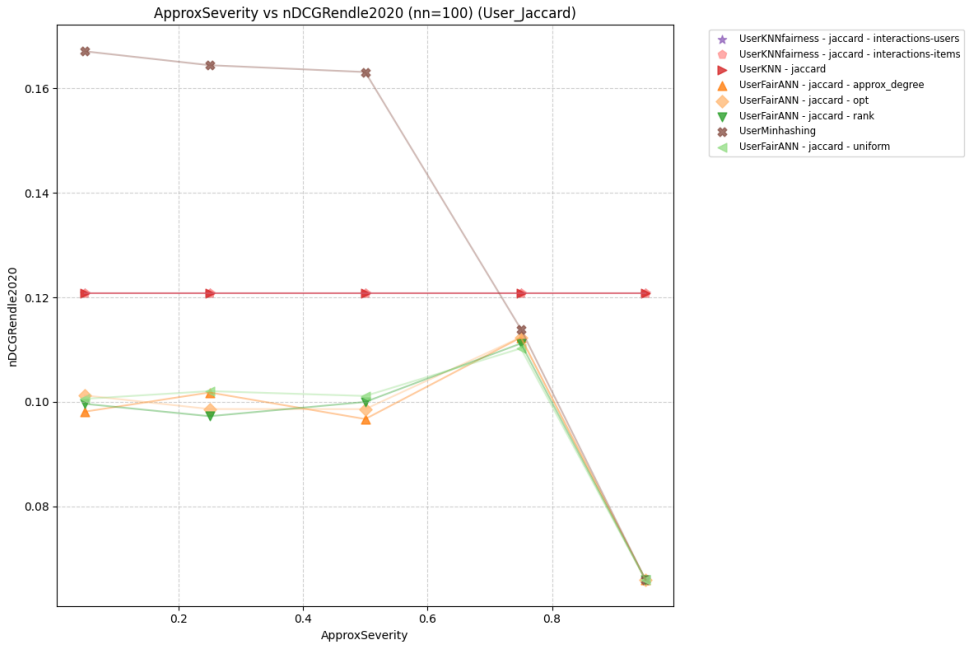
Approximation Severity vs nDCG - Item



- Annoy – best perfomances
- FairANN – bell curve
- MinHashing – drops with stricter threshold

Approximation Severity vs nDCG - User





- Annoy – excellent accuracy
- FairANN – drops as threshold rises

Accuracy results

- Similar results with different *k*-neighbor

Annoy

- Good accuracy compared with kNN baseline

FairANN

- Approximation sweet spot at **0.75**
- Poor accuracy at extremes due to **noise** and excessive **strictness**

MinHashing

- Performance follows increase in approximation

What is CR?

Measures **empirical efficiency**
How much the search space has been *pruned*

$$CR = \frac{\left|\left|M_s\right|\right|_0}{I^2}$$

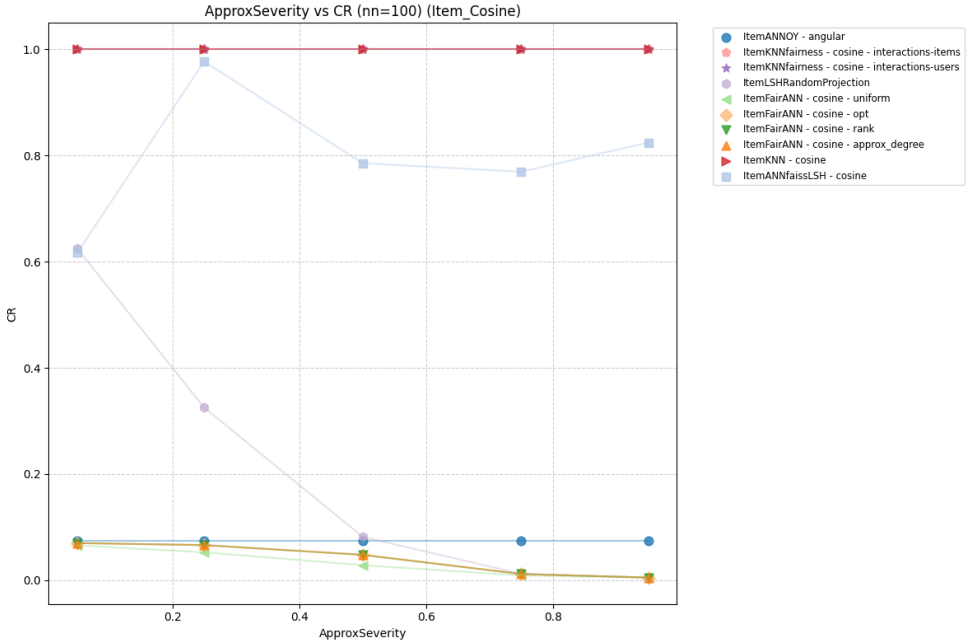
Too high CR

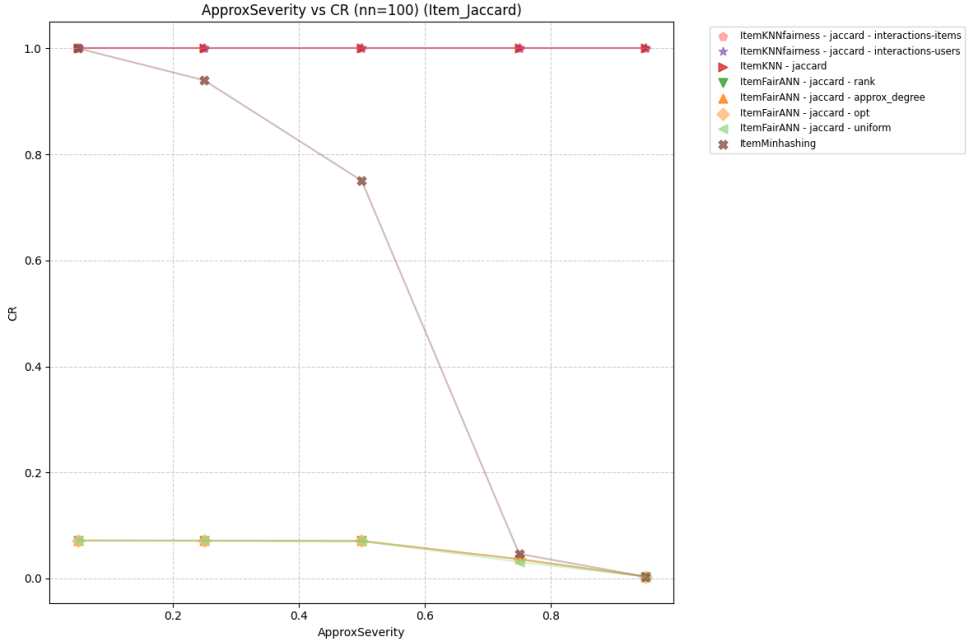
- Very poor effinciency – compares everything
- Certainly finds **true neighbors**

Too low CR

- Extremely efficient
- Barely looks at anything – **less accurate**

Approximation Severity vs CR - Item





- FairANN and Annoy – higher overall efficiency
- MinHashing - more efficient with stricter threshold
- Similar results for User based models

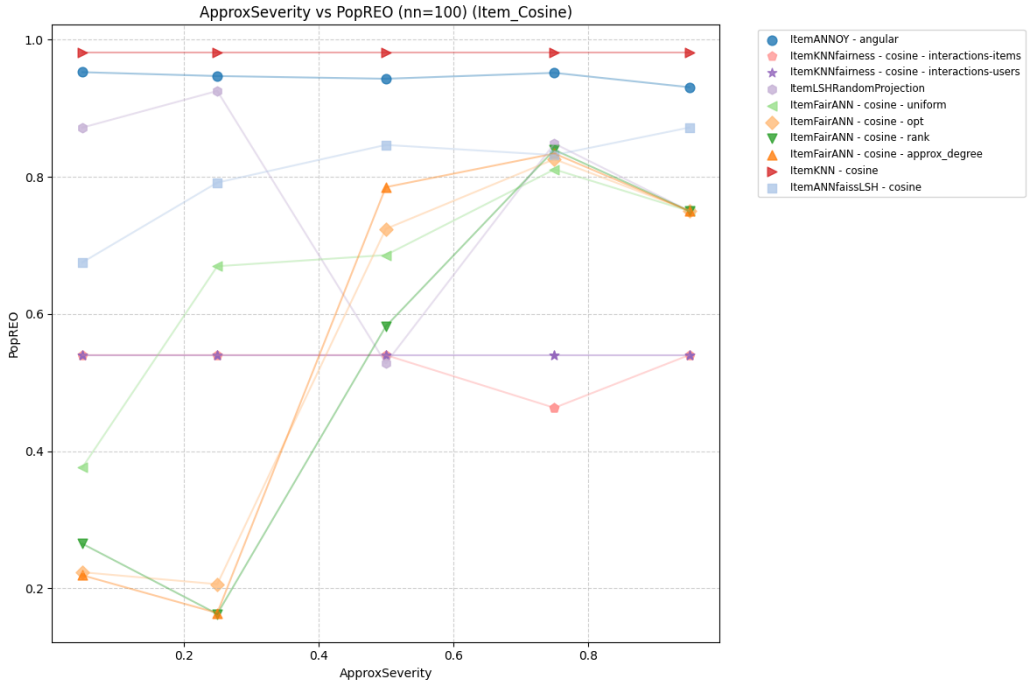
RQ1 – Results

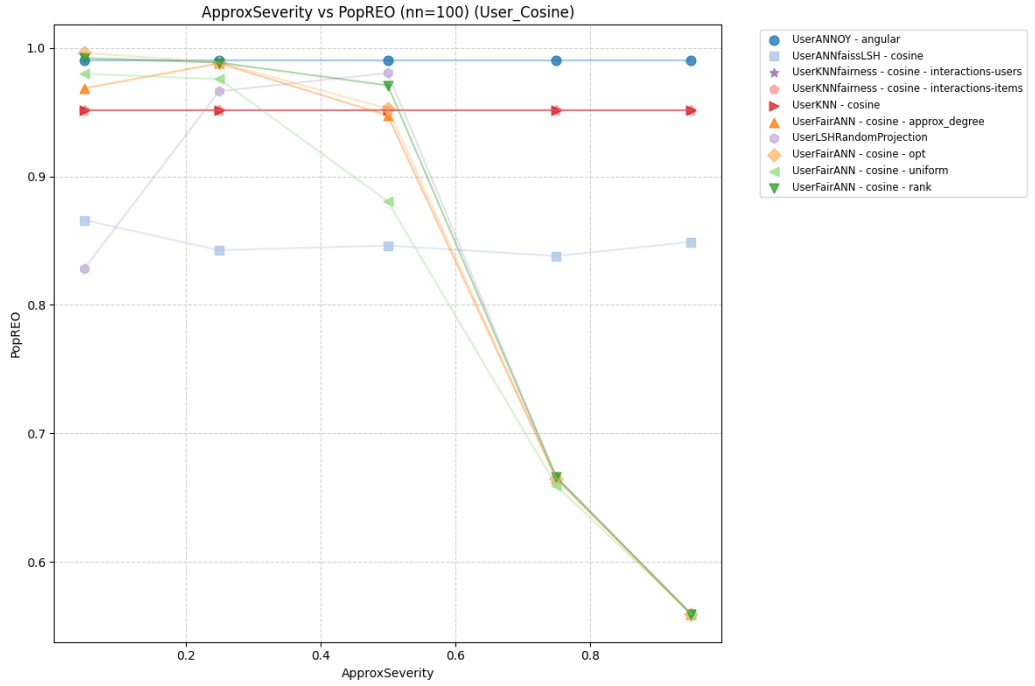
Accuracy	Introducing approximation causes drop in **accuracy**
Efficiency	10 times faster results

RQ2

PopREO – Equal Opportunity

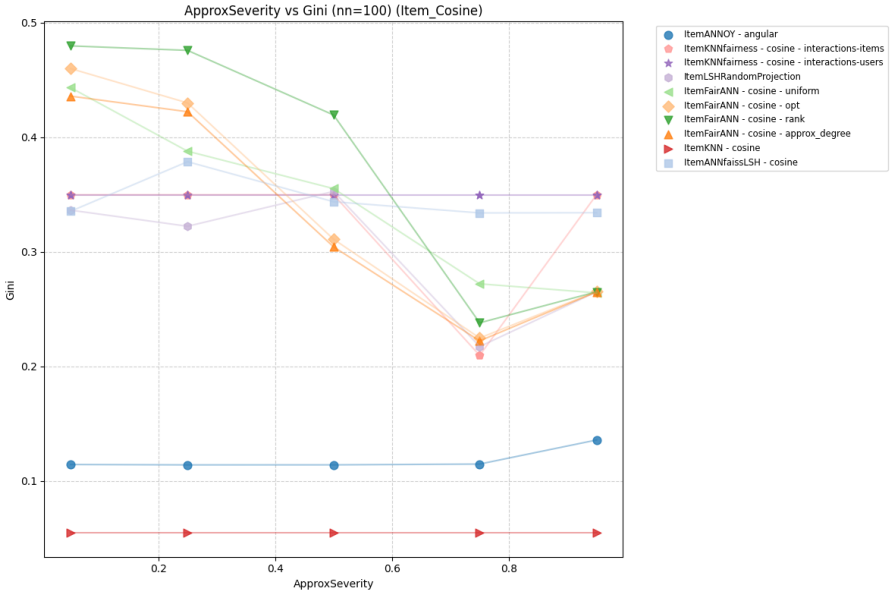
Item fairness – an item's chance of being recommended based on its relevance rather than popularity
Less **fair** with higher approximation – **data sparsity**

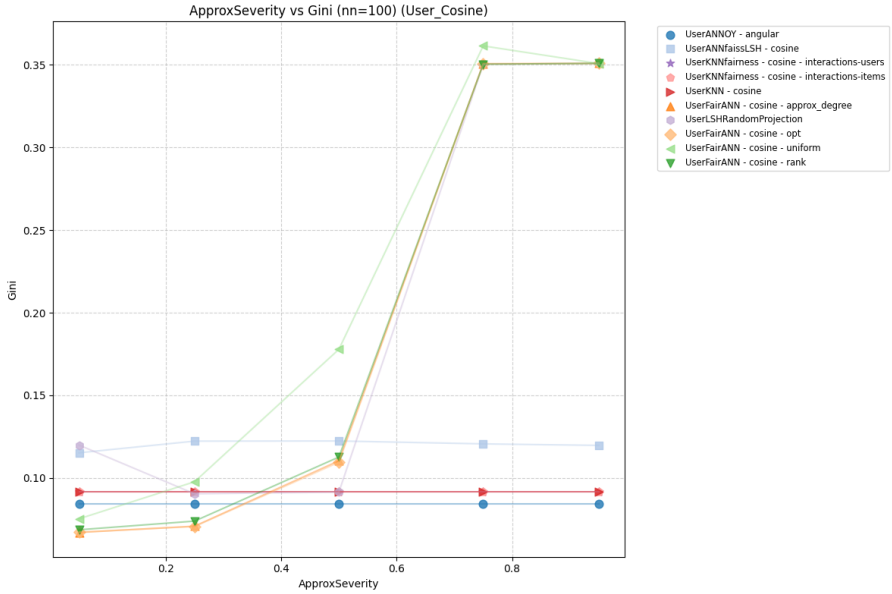




Gini Index

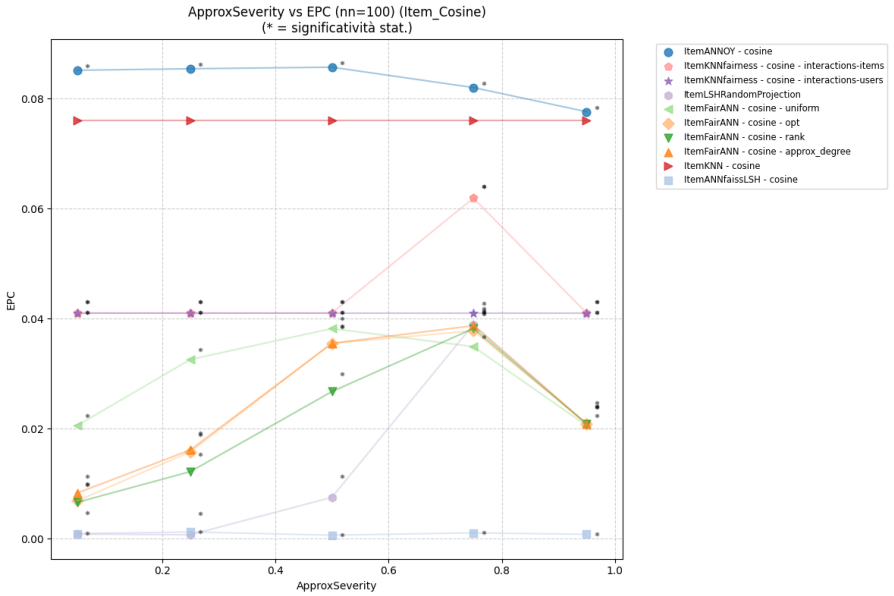
- Measures if the system is equally recommending the items in the catalog
- As strictness rises, **diversity** improves for user-based models and drops for item-based ones

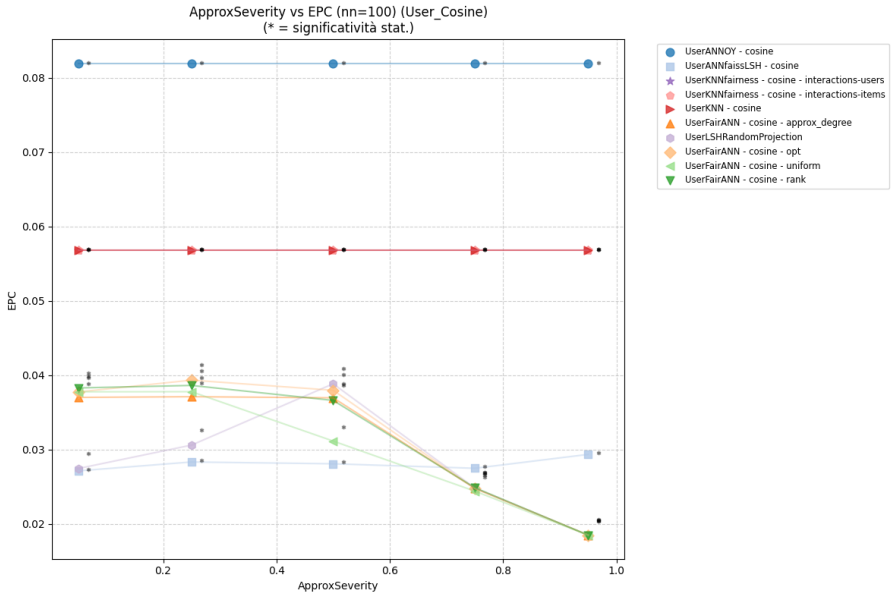




EPC

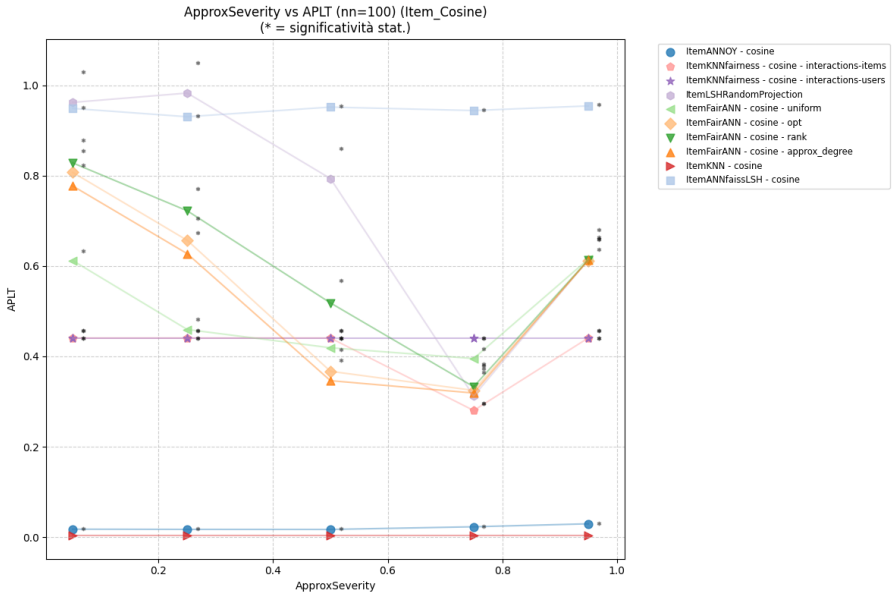
- Measures capability of recommending **niche** items
- Approximation inhibts **novelty**

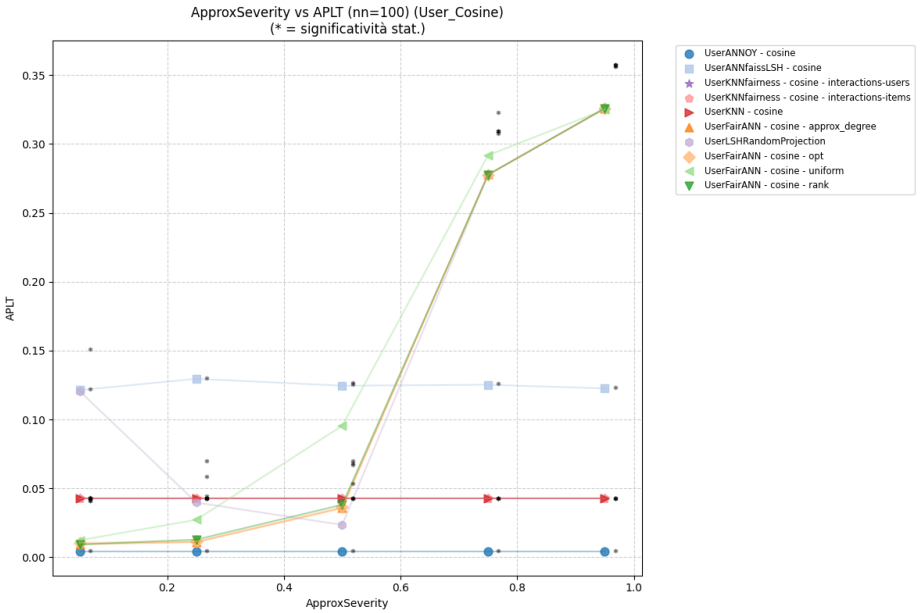




APLT

- Percentage of recommended items beloging to the long-tail
- Annoy has the best overall fairness, FairAnn performs best at $t \approx 0.75$





RQ2 – Results

- kNN models suffer from **popularity bias**: the noise introducted by ANN models makes recommendation **more fair**
- Approximation inhibits **novelty** due to focus on denser areas.
  Annoy is an exception for its intrinsic **randomness**
- Asymmetry due to different **pre-filtering strategies**:
  - User-based pre-filtering to preserve actually **active users**
  - No Item-based pre-filtering to preserve **catalog integrity**

RQ3

Hypervolume

- Volume-based quality index that measures the performances of a model from multiple objectives simultaneously
- Computes the volume beneath the Pareto curve

4 scenarios

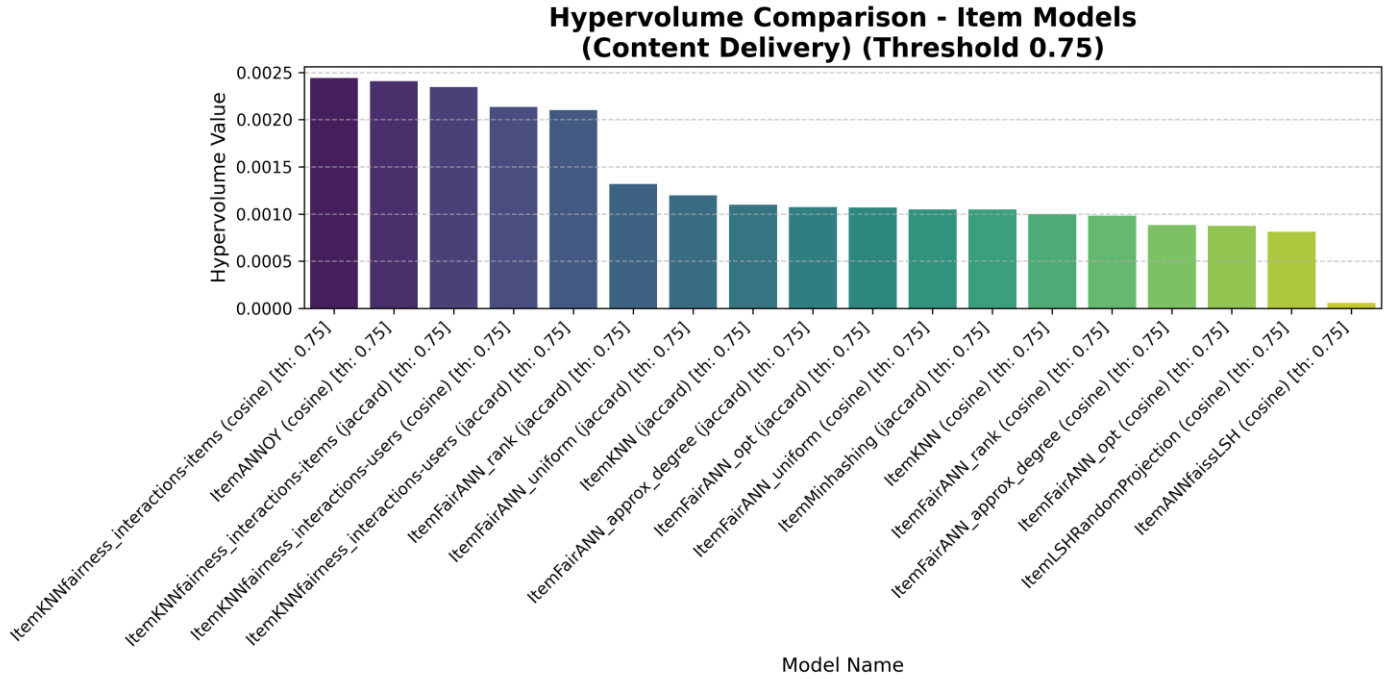
Content Delivery
nDCG - Gini - EPC

Content Exposure
nDCG - APLT

Efficiency
nDCG - CR

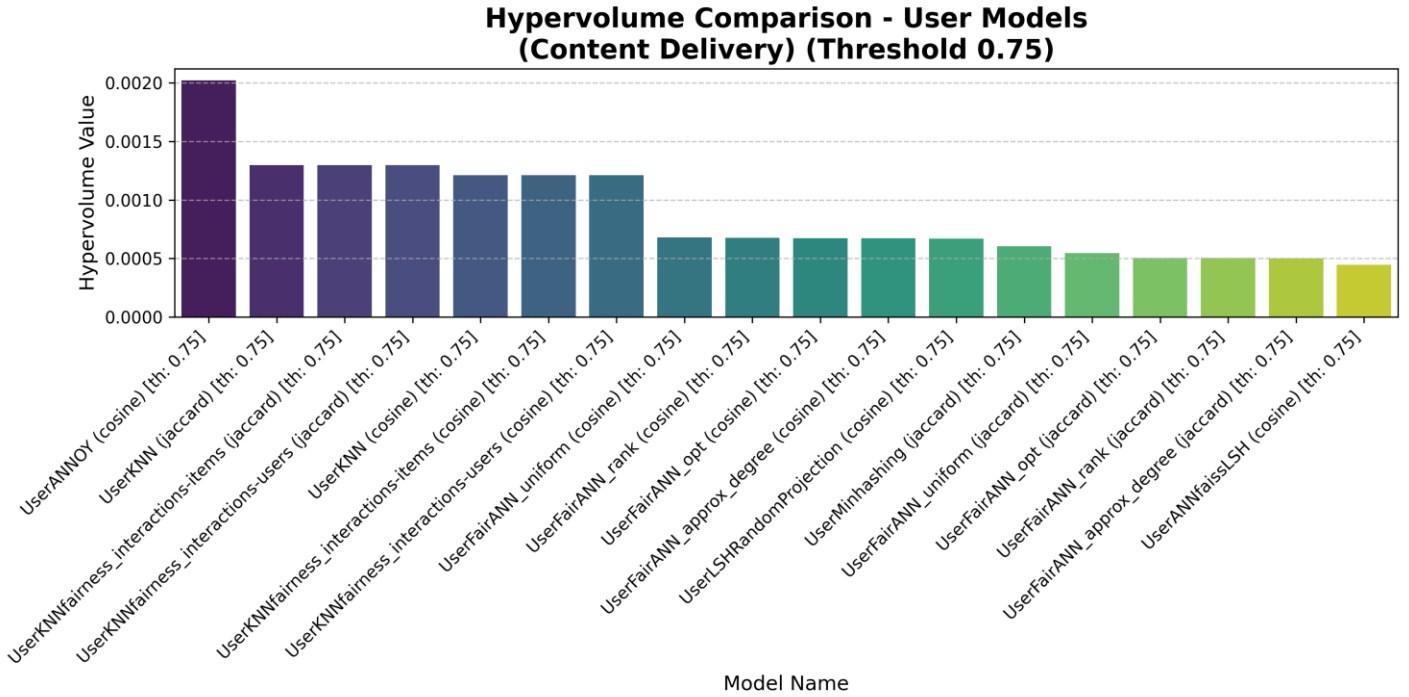
Mix
nDCG - PopREO - CR

Content Delivery



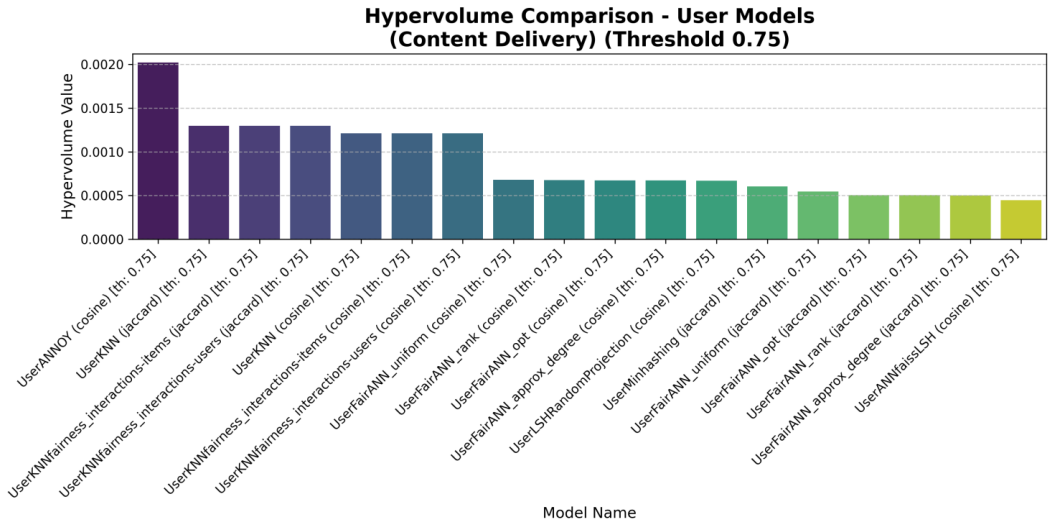
Algorithm	Similarity	Hypervolume
ItemKNNfairness_interactions-items	cosine	**0.002439**
ItemANNOY	cosine	**0.002407**
ItemKNNfairness_interactions-items	jaccard	**0.002345**

Content Delivery



Algorithm	Similarity	Hypervolume
UserANNOY	cosine	**0.002021**
UserKNN	jaccard	0.001298
UserKNNfairness_interactions-items	jaccard	0.001298

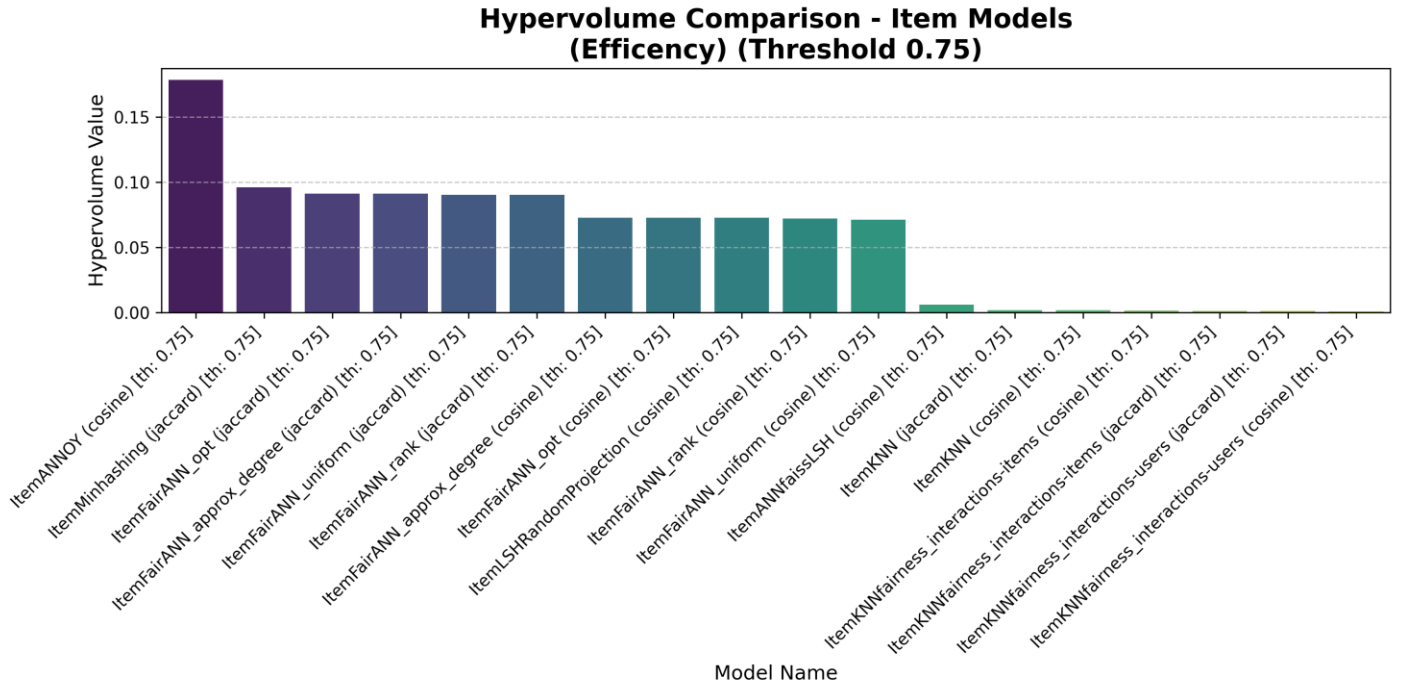
Content Delivery



Algorithm	Similarity	Hypervolume
UserANNOY	cosine	**0.002021**
UserKNN	jaccard	0.001298
UserKNNfairness_interactions-items	jaccard	0.001298

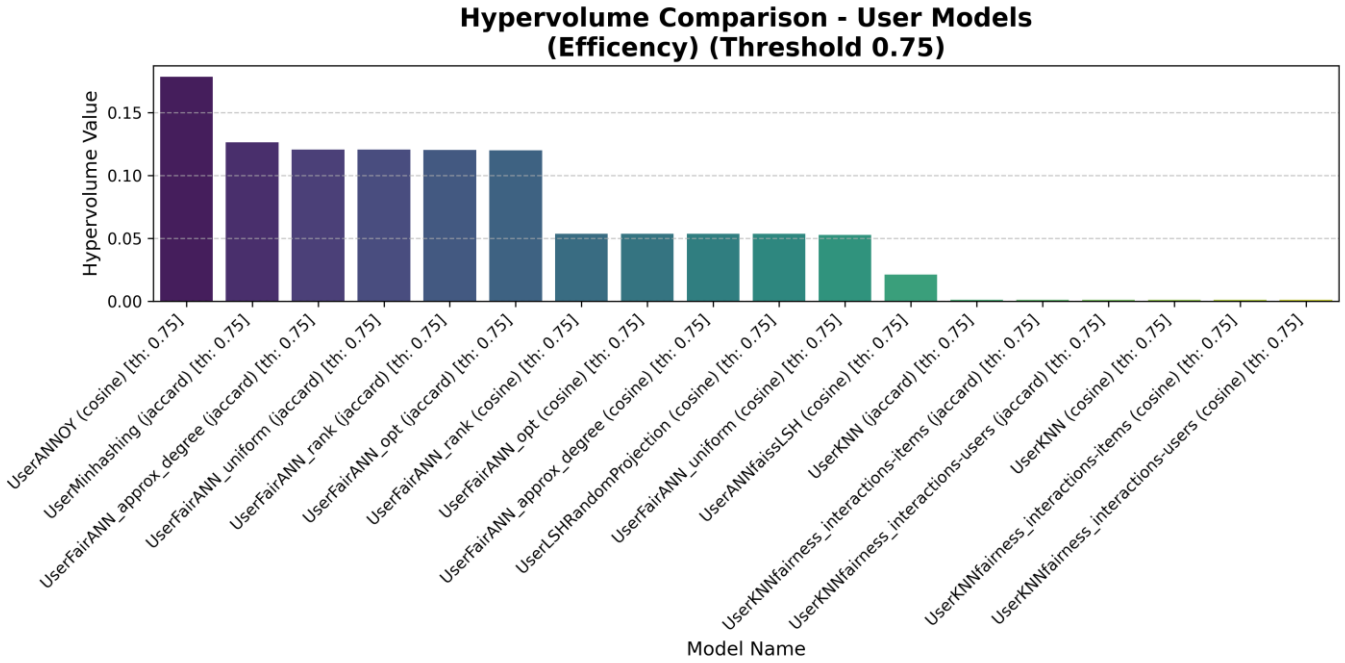
- Annoy displays the best diversity and novelty – both Item and User surpassing kNN models
- Good performances for kNN models – efficiency not taken into consideration

Efficiency



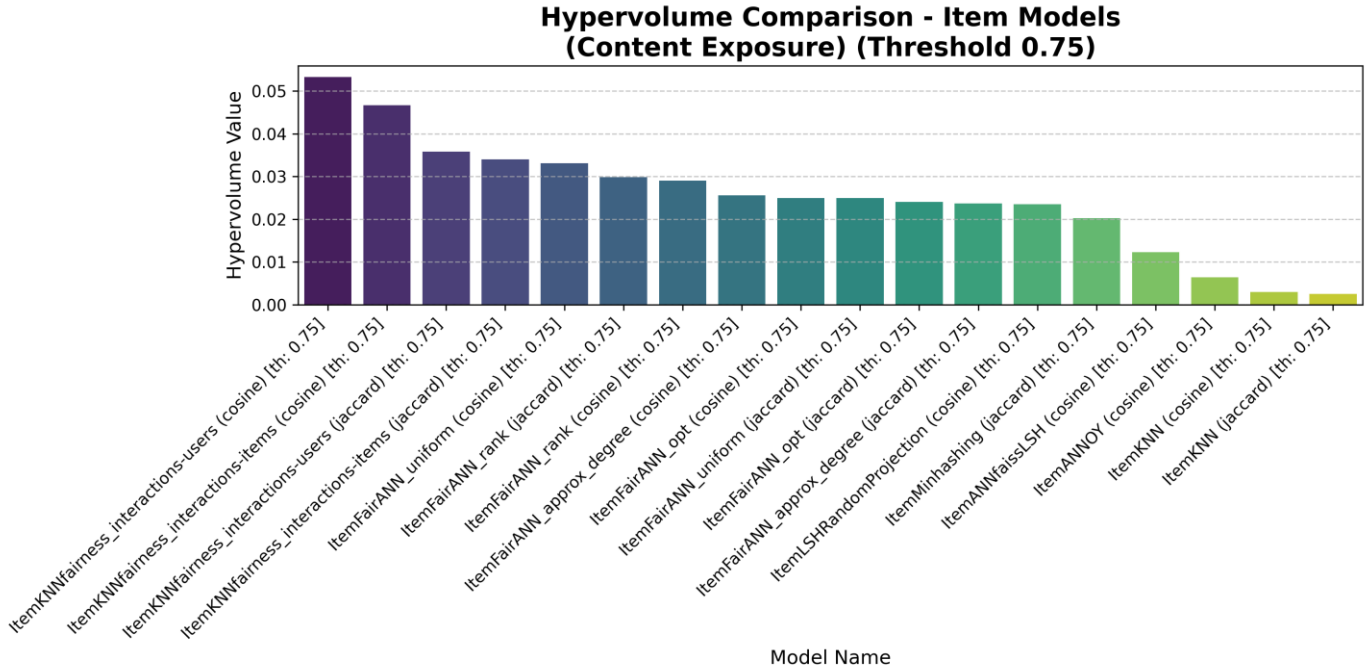
Algorithm	Similarity	Hypervolume
ItemANNOY	cosine	**0.178297**
ItemMinhashing	jaccard	**0.095867**
ItemFairANN_opt	jaccard	**0.091057**

Efficiency



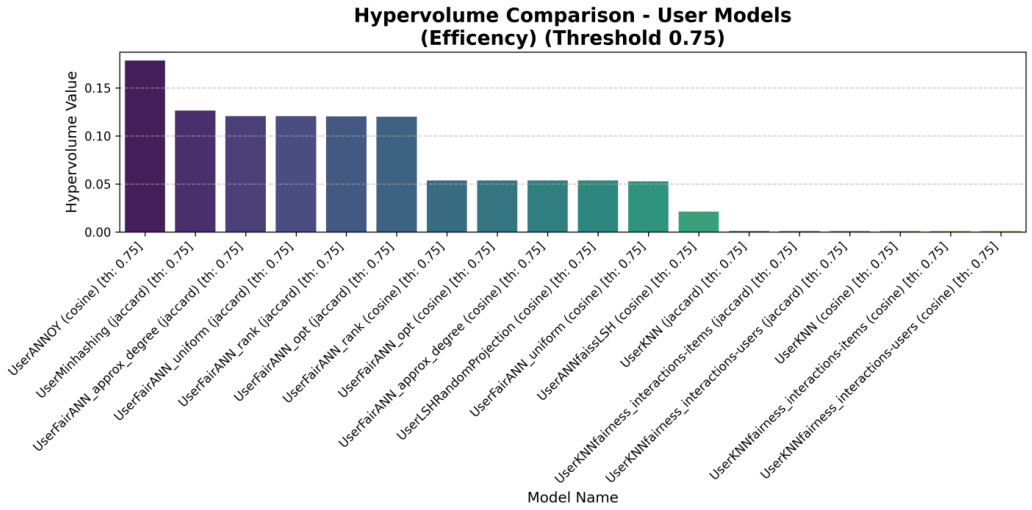
Algorithm	Similarity	Hypervolume
UserANNOY	cosine	**0.178329**
UserMinhashing	jaccard	**0.126362**
UserFairANN_approx_degree	jaccard	**0.120381**

Content Exposure



Algorithm	Similarity	Hypervolume
ItemKNNfairness_interactions-users	cosine	**0.053249**
ItemKNNfairness_interactions-items	cosine	**0.046695**
ItemKNNfairness_interactions-users	jaccard	**0.035808**

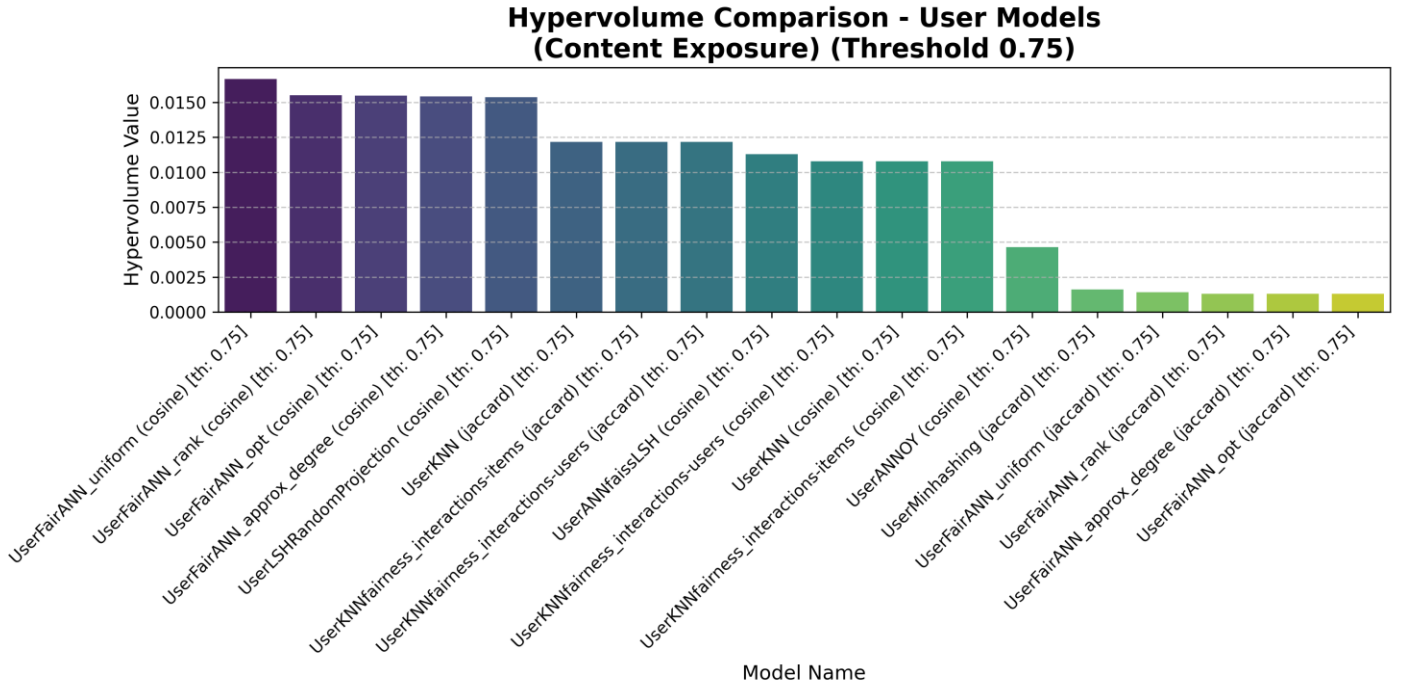
Efficiency



- Annoy offers the highest temporal efficiency mantaining **optimal accuracy**

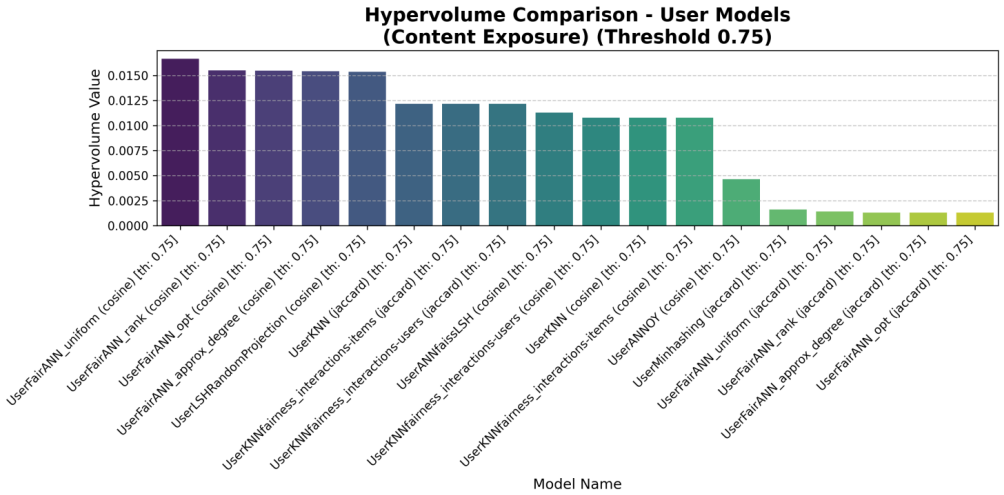
Algorithm	**Similarity**	**Hypervolume**
UserANNOY	cosine	**0.178329**
UserMinhashing	jaccard	**0.126362**
UserFairANN_approx_degree	jaccard	**0.120381**

Content Exposure



Algorithm	Similarity	Hypervolume
UserFairANN_uniform	cosine	**0.016660**
UserFairANN_rank	cosine	**0.015513**
UserFairANN_opt	cosine	**0.015477**

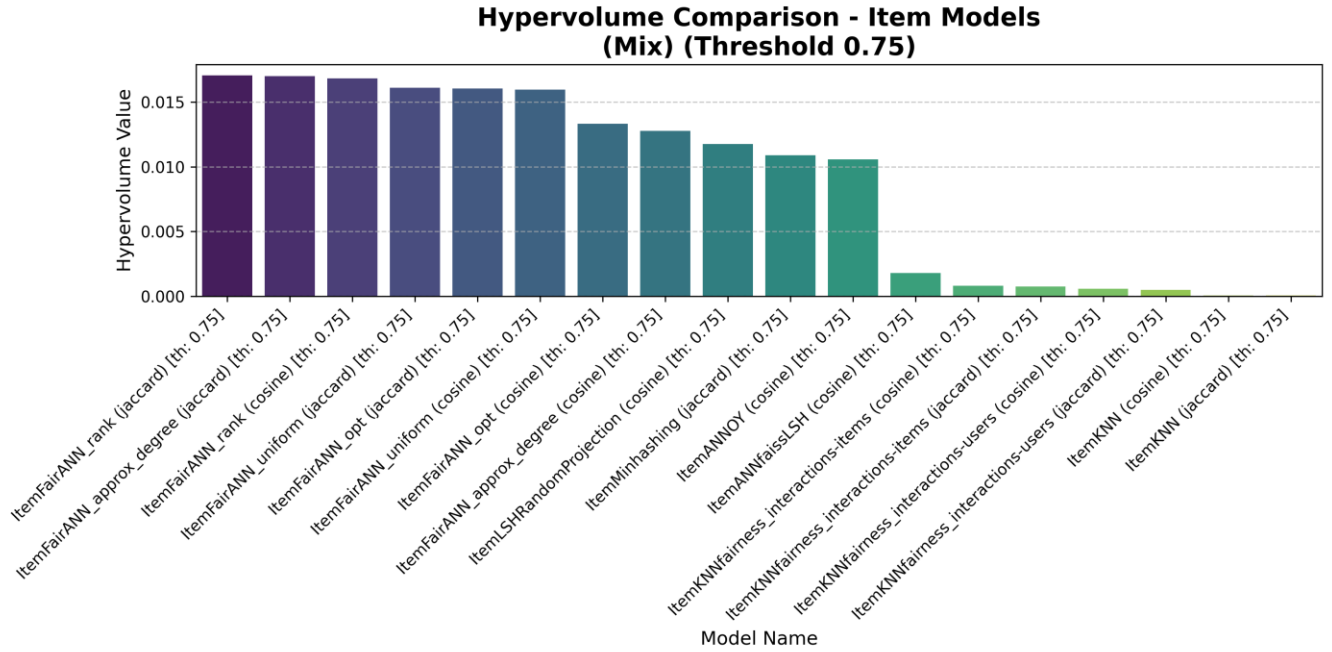
Content Exposure



- Fairness and FairANN model offer the best fairness – due to **sampling strategy**

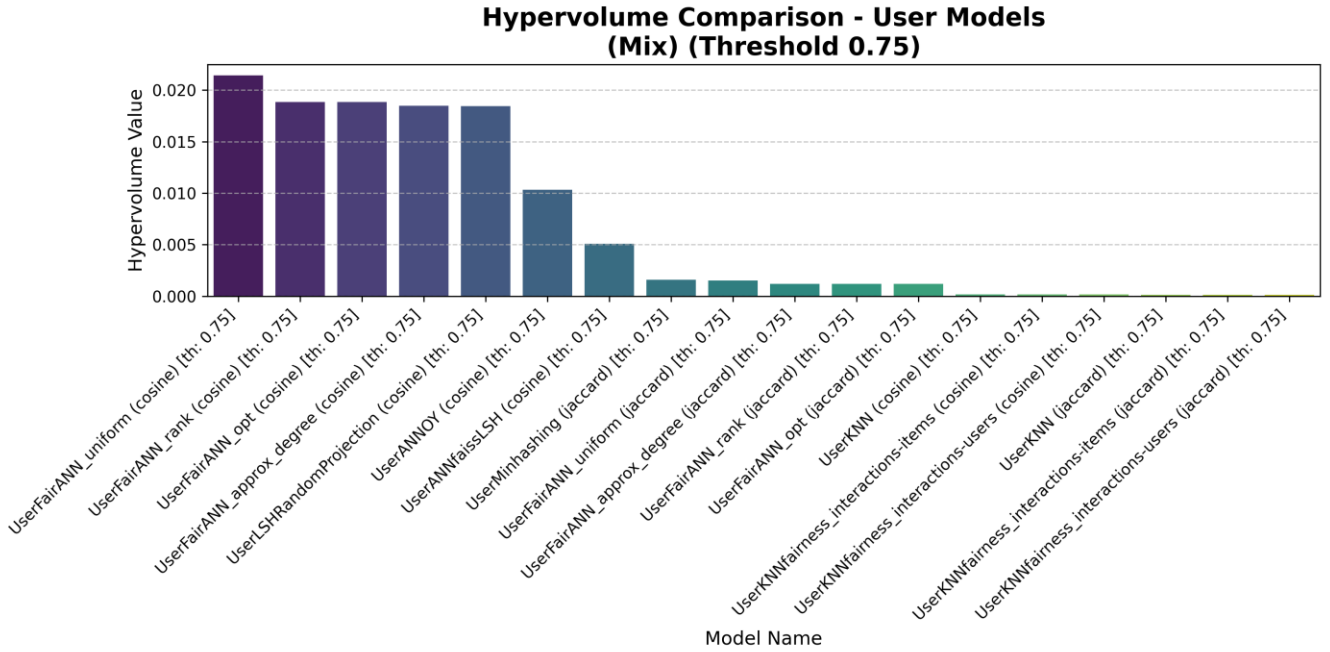
Algorithm	**Similarity**	**Hypervolume**
UserFairANN_uniform	cosine	**0.016660**
UserFairANN_rank	cosine	**0.015513**
UserFairANN_opt	cosine	**0.015477**

Mix



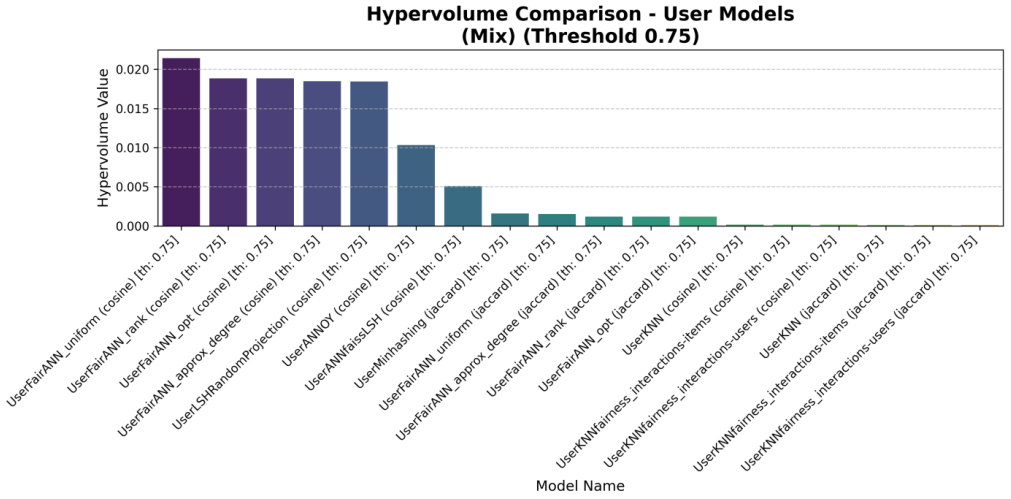
Algorithm	Similarity	Hypervolume
ItemFairANN_rank	jaccard	**0.017051**
ItemFairANN_approx_degree	jaccard	**0.017004**
ItemFairANN_rank	cosine	**0.016826**

Mix



Algorithm	Similarity	Hypervolume
UserFairANN_uniform	cosine	**0.021417**
UserFairANN_rank	cosine	**0.018846**
UserFairANN_opt	cosine	**0.018833**

Mix



- All ANN models are balanced
- kNN models suffer due to poor efficiency

Algorithm	**Similarity**	**Hypervolume**
UserFairANN_uniform	cosine	**0.021417**
UserFairANN_rank	cosine	**0.018846**
UserFairANN_opt	cosine	**0.018833**

Conclusion

- ANN models sacrifice a small quota in **accuracy** to gain a significant advantage in **efficiency**
- Absolute **accuracy** is not everything, in some scenario a **tradeoff** of accuracy and **beyound-accuracy** – novelty, discovery, fairness – and **efficiency** can be more appreciated
- These results on smaller dataset are encouraging enough to justify further experimentation on bigger dataset

Thank you for your attention!